HPCA 2027 Submission #NaN
Confidential Draft
Do NOT Distribute!!

QArchGauge: Architecture-Guided Modeling of Practical Quantum Advantage

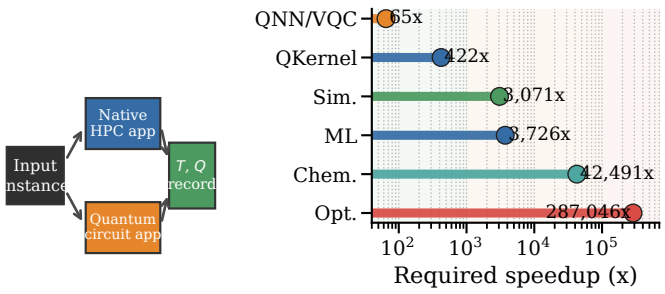
***Abstract*—Problem. Practical quantum advantage is often discussed as if faster quantum hardware will solve many applied problems automatically. This is not enough for computer architects. A useful claim must identify which future-system lever actually changes the outcome: logical gate latency, shot parallelism, error-correction overhead, readout/control latency, algorithmic quality, or the native HPC baseline. Without that decomposition, a speedup number does not tell architects where a quantum system should improve.**

Approach. We present QArchGauge, a measurement and architecture-projection framework for this break-even test on a Top-20 TOP500 leadership-scale HPC system. QArchGauge runs native baselines and quantum-circuit versions of the same workload with shared inputs and quality targets. It then converts measured circuit cost, quality gap, native runtime, circuit metadata, and repeated-execution structure into an advantage threshold over architecture variables: execution speed, shot throughput, quality recovery, and non-kernel overhead.

Result. On the leadership-scale system, our expanded `sklearn digits` sweep runs 160 cases across class pairs, sample counts, PCA dimensions, depths, and seeds. Quantum kernels require a median 421.9× speedup to match the native path; QNN/VQC requires 64.9× but usually misses native quality. We then run a 3,552-case practical suite across ML, chemistry, optimization, and simulation on up to 256 GPUs, plus a 104-case OpenFermion/PySCF chemistry coverage gate. Median required speedups range from 3,071.0× for simulation to 287,045.6× for optimization. The architecture implication is workload-specific: at 90% quality-gap recovery, $10^4\times$ projected execution improvement covers 54.9% of simulation cases, while $10^5\times$ covers 57.1% of chemistry cases; ML and optimization remain primarily quality-limited. These results do not claim quantum wins today. They show where future quantum architectures must spend effort before speed becomes useful.

I. Introduction

Quantum computing is often described through broad application promises: better machine learning, faster molecular modeling, improved optimization, and natural Hamiltonian simulation. For computer architects, the question is more concrete. Which future quantum-system resource should improve first: logical gate latency, two-qubit fidelity, shot throughput, readout/control latency, memory/communication near the controller, or algorithmic quality? A future quantum path is useful only if it beats a strong native HPC path on the same input, at the same quality, after all application overheads are included. Current quantum hardware is not yet large and reliable enough to answer this question directly for many practical workloads. HPC systems therefore play a second role beyond accelerating quantum-circuit simulation: they are the measurement instrument for modeling which architecture improvements would make a quantum-circuit application competitive.



(a) Comparison path. (b) Measured thresholds.

Fig. 1: Application-level comparison target. The key comparison is not simulator versus simulator. The key comparison is native application execution versus quantum-circuit application execution and projected quantum hardware execution.

Existing simulation systems provide the first part of this stack. Libraries such as `cuQuantum` execute quantum circuits on GPUs, while prior systems such as ScaleQsim and AURORA-Q improve simulation scalability on leadership-scale systems [1], [2]. These systems answer an important question: how can we simulate larger quantum circuits faster on HPC? However, simulator scalability is not the same as application-level quantum advantage. A simulator can be faster than another simulator, but the quantum-circuit application can still be slower than a native HPC application because encoding, circuit generation, repeated evaluations, measurement, optimizer steps, and postprocessing remain in the end-to-end path.

Figure 1 shows the comparison target of QArchGauge. A circuit simulator is only one component of the quantum path. The full quantum path also includes application stages that do not disappear when the circuit kernel becomes faster. Three points follow. First, simulator speed is not application speed: measuring only `cuQuantum` time can hide encoding, measurement, optimization, and classical control. Second, native baselines are an architecture constraint because native HPC/ML applications already use mature software stacks and efficient kernels. Third, advantage is a design-space boundary: the goal is to invert today's measurement and compute the required quantum gate time, shot throughput, error overhead, and quality recovery for future hardware.

TABLE I: Positioning of QARCHGAUGE relative to prior work.

Study/System	HPC sim.	Native baseline	Cost model	HW projection	Arch. bottleneck
ScaleQsim/AURORA-Q	✓				
`cuQuantum`	✓				
Benchmark suites			partial	partial	partial
Native baselines		✓			
QARCHGAUGE	✓	✓	✓	✓	✓

The same issue becomes stronger when the workload moves beyond one small ML example. Public claims about quantum computing often jump from one successful example to broad expectations such as better AI, faster drug discovery, or better optimization. A systems paper cannot validate such claims with one toy workload. It must cover representative application families and show the threshold for each family. For a practical workload, quantum advantage depends on several conditions being true at the same time: the quantum path must reach the target quality, the native baseline must be strong, the circuit must fit the available logical qubits, shots must be parallelized enough, and error overhead must not erase the speedup. QARCHGAUGE therefore reports an advantage frontier rather than a single yes/no answer.

Many previous studies focus on one part of this stack. Table I summarizes the gap. ScaleQsim and AURORA-Q improve quantum simulation on HPC systems. `cuQuantum` provides high-performance GPU primitives. Benchmark suites such as SupermarQ, QASMBench, MQT Bench, and QAOAKit improve coverage and reproducibility [3]–[6]. Native ML frameworks provide strong classical baselines. QARCHGAUGE connects these pieces into an application-level architecture requirement model: it reports whether the next useful improvement should be faster logical operations, more shot parallelism, lower error overhead, better quantum output quality, or a different algorithm.

Key Idea. QARCHGAUGE treats practical quantum advantage as an architecture design-space problem. The advantage threshold is the point where the projected quantum-circuit path becomes faster than the measured native HPC and ML path under the same workload and quality target. For each workload, QARCHGAUGE records the cost of each stage and then projects the quantum hardware region where this condition holds. The output is not one global speedup number; it is a bottleneck label that says whether future systems should prioritize gate throughput, shot parallelism, error/quality recovery, host-control overhead, or a stronger quantum algorithm.

We present QARCHGAUGE, an architecture-guided quantum-advantage modeling and analysis study for HPC systems. QARCHGAUGE starts with a controlled ML workload: `sklearn digits` native ML baselines, quantum kernel classifiers, and QNN/VQC classifiers. This first workload validates the measurement pipeline. We then scale the study to a practical suite with ML, molecular energy estimation, combinatorial optimization, and Hamiltonian simulation. The evaluation first exposes the bottleneck, then studies baseline sensitivity, scaling behavior, hardware projection, and workload-specific advantage frontiers.

This paper makes the following contributions:

- **Architecture threshold model.** We define practical quantum advantage as a same-task, same-quality break-even condition between a native HPC path and a quantum-circuit application path, then express the result as future-system requirements.
- **Logging framework.** We build a pipeline that records runtime breakdowns, quality, circuit metadata, GPU metadata, Slurm metadata, and repeat-timing evidence.
- **Practical workload evidence.** We evaluate controlled quantum ML and a suite covering ML, chemistry, optimization, and simulation, including a 3,552-case large profile, OpenFermion/PySCF chemistry, and sparse Krylov baselines.
- **Scale-out validation.** We run the large profile up to 256 GPUs and report weak scaling, fixed-work scaling, bundled allocation behavior, and repeat timing.
- **Architecture-region analysis.** We convert measured circuit costs into required execution speed, shot throughput, and quality-gap recovery, then classify whether each workload is speed-limited, quality-limited, shot-limited, encoding-limited, or native-dominated.

II. BACKGROUND

A. Quantum-Circuit Application Paths

Quantum-circuit applications map input data or problem instances into a sequence of quantum gates. In quantum machine learning, common paths include quantum feature maps, quantum kernels, and variational quantum circuits. A feature-map pipeline encodes classical features into circuit parameters and extracts observables or kernel values. A QNN/VQC pipeline additionally optimizes trainable circuit parameters through repeated circuit evaluations.

Repeated Execution. The key systems implication is repetition. A single circuit execution is rarely the full application. Training and evaluation require many samples, classes, optimizer steps, shots, and circuit variants. Therefore, the cost of a quantum application must include encoding, circuit construction, repeated simulation or execution, measurement, loss computation, and classical postprocessing.

Practical Application Families. QARCHGAUGE treats the first ML workload as a calibration workload, not as the whole target. Practical quantum claims usually appear in four application families. First, quantum ML claims include classification, kernel methods, and variational classifiers. Second, chemistry and drug-discovery claims include molecular energy estimation, Hamiltonian simulation, and candidate screening loops. Third, optimization claims include portfolio, routing, scheduling, and QAOA-style objective minimization. Fourth, scientific simulation claims include quantum dynamics and materials workloads. These families have different native baselines and different quantum costs. A useful advantage model must therefore report a threshold per workload family, not one global number for all quantum computing.

Terminology. The project name is QARCHGAUGE, and the paper's technical object is a *practical quantum-advantage threshold*. This is not a claim that current quantum hardware wins on the measured applications. A threshold is the break-even condition under which the projected quantum-circuit

application path becomes faster than the selected native HPC path while meeting the same quality target. We use *advantage region* for the set of speed and quality-recovery assumptions that satisfy this condition, and *architecture bottleneck* for the resource that prevents a case from entering that region. This distinction is important because a circuit simulator can be useful for measurement even when both the simulator and the projected near-term quantum path are far from application-level advantage.

B. Native Baselines and Threshold Model

Native HPC/ML baselines execute the same task directly on CPUs or GPUs. For the first QARCHGAUGE workload, the native path uses `sklearn digits` with PCA and classical classifiers such as logistic regression, MLP, and optionally SVM/RBF. These baselines are intentionally strong and simple: if a quantum-circuit model cannot beat a well-optimized native baseline at the same target quality, it does not satisfy the practical advantage threshold.

Leadership-scale GPU Platform and cuQuantum. The measurement platform is a Top-20 TOP500 leadership-scale HPC system with GPU nodes suitable for quantum-circuit simulation through `cuQuantum` and cuStateVec [7]. Our environment uses an existing `cuQuantum` installation and validates it through login-node smoke tests before any allocation-consuming job. Login-node tests are used only for correctness and pipeline validation. Performance experiments run through Slurm and record queue time, elapsed time, allocation resources, and charged node-hours.

Break-even Condition. For a fixed task, dataset split, target quality, and system budget, the native application path is:

$$T_{\text{native}} = T_{\text{input}} + T_{\text{preprocess}} + T_{\text{train}} + T_{\text{eval}} + T_{\text{postprocess}}. \quad (1)$$

The quantum-circuit path simulated on `cuQuantum` is:

$$T_{\text{qc\_sim}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{circuit}} + T_{\text{cuQuantum}} + T_{\text{measure}} + T_{\text{postprocess}}. \quad (2)$$

The projected quantum hardware path is:

$$T_{\text{qhw}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{compile}} + T_{\text{execute}} + T_{\text{measure}} + T_{\text{error}} + T_{\text{postprocess}}. \quad (3)$$

The condition for projected quantum advantage is:

$$T_{\text{qhw}} < T_{\text{native}} \quad (4)$$

at the same target quality. An architecture study then asks which term must change to satisfy this inequality: native baseline strength, logical execution time, shot parallelism, error/control overhead, or quantum output quality.

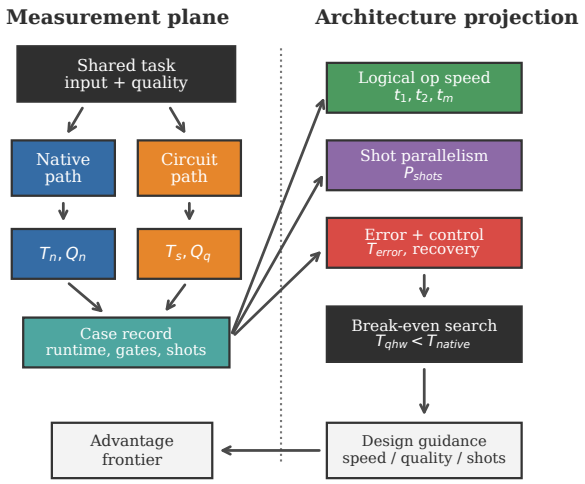


Fig. 2: Architecture and procedure of QARCHGAUGE.

III. QARCHGAUGE DESIGN

Overview of QARCHGAUGE's Design. QARCHGAUGE is designed to answer one architecture question: which future quantum-system resource must improve before a quantum-circuit application can beat the native HPC application for the same task? Figure 2 summarizes the execution path, and Figure 3 shows how measurements become a bottleneck decision. First, QARCHGAUGE fixes the input and quality target. It then measures a native path and a quantum-circuit path, records the circuit metadata, and projects the hardware region where the quantum path would break even.

Design Boundary. QARCHGAUGE is not a new quantum circuit simulator, compiler, or quantum algorithm. It treats existing simulators and native methods as execution engines behind a controlled workload interface. The design contribution is the measurement discipline and projection interface: same input, same quality metric, strongest valid native path, measured quantum-circuit path, and explicit architecture bottleneck classification. This boundary is important because faster simulation alone does not answer whether a future quantum computer would beat the native application or which architecture knob would make the difference.

A. Overall Procedure

Figure 2 shows the overall procedure. QARCHGAUGE has two main phases: *Measurement* and *Advantage Analysis*.

Measurement. When an experiment starts, QARCHGAUGE reads the workload configuration. The configuration includes the dataset, train/test split, feature dimension, model type, circuit type, shots, and target quality. QARCHGAUGE then runs the native path and the quantum-circuit path using the same input data. This prevents unfair comparisons caused by different preprocessing or different quality targets.

Config Record. Before any path is executed, QARCHGAUGE materializes a single configuration record. This record is the

logical owner of the experiment. It stores the workload family, instance ID, random seed, feature transform, native candidates, quantum circuit family, circuit depth, shot setting, and quality metric. The native and quantum paths do not choose these values independently. They receive the same record and append path-specific measurements to it. This mirrors the task-oriented style in the previous papers: the system first creates a stable representation of the work, then later components operate on that representation.

Advantage Analysis. After measurement, QARCHGAUGE reads the circuit metadata and runtime breakdown. It extracts the number of qubits, circuit depth, one-qubit gates, two-qubit gates, shots, and optimizer iterations. Using this information, QARCHGAUGE computes the quantum hardware speed required to make the projected quantum path faster than the measured native path.

Architecture Variables. The projection plane exposes four groups of future-system variables. The first group is *logical execution speed*: one-qubit gate latency, two-qubit gate latency, measurement latency, and controller scheduling overhead. The second group is *shot parallelism*: how many independent circuit evaluations can run concurrently without exceeding the device, decoder, or queue budget. The third group is *quality recovery*: algorithmic improvement, error mitigation, or fault tolerance that reduces the measured output gap. The fourth group is *host and data overhead*: encoding, compilation, optimizer feedback, and postprocessing that remain outside the quantum kernel. QARCHGAUGE keeps these variables separate so an advantage frontier can say whether an architecture needs faster logical gates, more parallel shots, lower error overhead, or a better algorithm before speed matters.

Microarchitecture Map. The variables are intentionally architecture-facing. The runtime overhead term is not a single magic constant; it can be expanded as

$$T_{\text{error}} = T_{\text{compile}} + T_{\text{map}} + T_{\text{route}} + T_{\text{decode}} + T_{\text{ms}} + T_{\text{io}} + T_{\text{cq}} + T_{\text{queue}},$$

where the terms capture compilation, logical placement, routing, decoder latency, magic-state preparation/injection, data loading or QRAM-style access, classical-quantum feedback, and queueing or device contention. Likewise, the useful shot rate is bounded by the slowest system resource:

$$P_{\text{shots}}^{\text{eff}} = \min(P_{\text{device}}, P_{\text{decode}}, P_{\text{control}}, P_{\text{queue}}, N_s N_{\text{eval}}).$$

This map is the connection between the measured application record and a future quantum computer architecture. A surface-code study can plug in code distance, cycle time, decoder bandwidth, and magic-state factory throughput; a control-system study can instead change the control and feedback terms. The same measured case then moves on the same frontier.

Simulator-Hardware Separation. The measured `cuQuantum` runtime is not a physical quantum latency. It executes the quantum-circuit application path, produces output quality, and records circuit metadata; the hardware projection then prices the same case with logical counts, repeated evaluations, effective shots, and explicit overhead terms. Thus state-vector memory behavior, Python process cost, and GPU setup time are measurement-stack evidence, not future-QPU schedule terms.

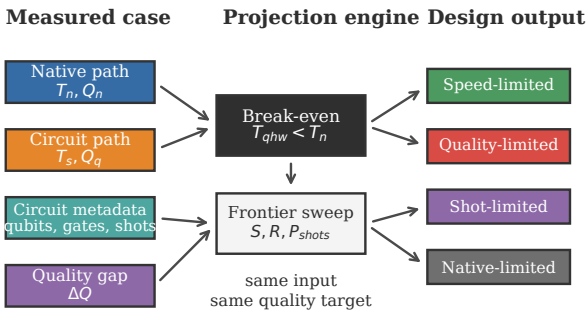


Fig. 3: Projection flow in QARCHGAUGE. Measurements are not the final answer; they are inverted into speed, quality, shot, and native-baseline requirements.

The important output is a bottleneck label, not another timing table. A speed-limited case points to logical-operation throughput or shot parallelism. A quality-limited case points to encoding, ansatz, noise, mitigation, or fault-tolerance quality. A native-limited case says that the classical software stack moved the break-even target and the quantum algorithm must change.

Failure handling. QARCHGAUGE treats failed paths as measurement results rather than silently dropping them. If a native candidate fails, the record keeps the error and continues with other native candidates. If a quantum path fails, the record keeps the circuit metadata available up to the failure point and marks the case as non-advantaged. This rule is important for large sweeps because feasibility is part of the systems question. A workload that only succeeds after removing difficult cases would overstate the advantage frontier.

B. Shared Workload Control

Dataset control. QARCHGAUGE first fixes the workload. The first full workload is `sklearn digits`. QARCHGAUGE uses a fixed train/test split and applies PCA to 4, 8, 12, and 16 dimensions. The same reduced features are used by every native and quantum model.

Instance identity. Each measured case receives a deterministic identity. For ML, the identity includes the class pair, sample count, PCA dimension, circuit depth, and seed. For chemistry, it includes the molecule, basis, active-space setting, and circuit layer count. For optimization, it includes the graph family, graph size, seed, and QAOA layer count. For simulation, it includes the Hamiltonian model, number of qubits, evolution time, and approximation setting. This identity is used in filenames, JSON records, summaries, and paper figures. As a result, a figure bar can be traced back to the exact raw case that produced it.

Application control. QARCHGAUGE then extends the same rule to practical workloads. For drug discovery, the native path

may be a classical molecular simulation or learned surrogate, while the quantum path may be a VQE-style molecular energy circuit. For optimization, the native path may be a mixed-integer, local-search, or GPU heuristic baseline, while the quantum path may be a QAOA-style circuit. For simulation, the native path may be a classical numerical solver, while the quantum path may be a Hamiltonian simulation circuit. In every case, QARCHGAUGE keeps the task, input instance, output metric, and quality target fixed.

Quality control. QARCHGAUGE defines the target quality before execution. The target can be a fixed accuracy threshold or a time-to-best-quality curve. This is important because a quantum path with lower accuracy cannot be claimed faster than a native path with higher accuracy.

Quality normalization. Different application families expose different quality values, so QARCHGAUGE stores both the native metric and a tolerance-scaled gap. For higher-is-better metrics, the gap is the native score minus the quantum score. For lower-is-better metrics, it is the quantum error minus the native/reference error. The current paper uses $\epsilon_w = 0.02$ for ML/optimization accuracy or objective gaps and $\epsilon_w = 0.01$ for chemistry/simulation energy or observable gaps. These are not domain-final budgets; they are deliberately small thresholds for sensitivity analysis and are recorded in the projection artifacts. QARCHGAUGE compares the residual gap to the workload tolerance ϵ_w:

$$(1 - R)\Delta Q_w \leq \epsilon_w.$$

Here R is a hypothetical recovery factor. ΔQ_w is not a probability and is not bounded by one; values above one mean that the measured quantum path is more than one tolerance unit away from the target.

Why this matters. Without shared workload control, the comparison becomes ambiguous. A native model may use a different feature dimension or a different split from the quantum model. QARCHGAUGE avoids this by making the workload controller the first step in the pipeline.

C. Application Path Execution

1) Native Path Execution: **Classical Baselines.** The native path executes the best available non-quantum implementation for the same input. For ML, QARCHGAUGE supports softmax regression, MLP, linear ridge classification, RBF kernel ridge classification, k-nearest neighbors, and nearest-centroid baselines without requiring an external ML package. A separate ML strong-native gate instantiates PyTorch AMP CNN/MLP and XGBoost GPU-hist candidates on the same measured digits configurations to test whether a production GPU stack changes T_{native}. For chemistry, the native path computes molecular energy from the same Hamiltonian by exact dense diagonalization and, in the strengthened implementation, sparse Lanczos/eigensolver baselines for Pauli Hamiltonians. For optimization, QARCHGAUGE records exact enumeration when feasible and also greedy, local-search, and simulated-annealing baselines. For scientific simulation, the native path includes dense exact time evolution and a sparse Krylov-style evolution baseline. The native time used in the threshold is the fastest implemented model that reaches the best native quality within a small tolerance. This rule prevents a weak or low-quality native model from making the quantum path look artificially competitive.

Runtime breakdown. QARCHGAUGE records preprocessing time, training time, inference time, and total time. It also records accuracy and model metadata. These values define T_{native}, which is the target that the projected quantum hardware path must beat.

Baseline selection rule. The native path may produce several valid answers for one case. QARCHGAUGE first removes candidates that do not meet the family-specific quality rule. Among the remaining candidates, it selects the fastest one as the native baseline. If no native candidate reaches the quality target, the best-quality native candidate is still recorded, but the case is marked so that the threshold analysis does not create an artificial win. This selection rule is deliberately conservative: adding a stronger native method can only make the quantum threshold harder, not easier.

2) Quantum Kernel Path: **Feature Map Execution.** The quantum kernel path maps each PCA feature vector into a quantum feature map. `cuQuantum` simulates the circuit and produces state or observable data for kernel entries. A classical classifier then consumes the kernel matrix.

Cost breakdown. QARCHGAUGE records encoding time, circuit construction time, `cuQuantum` execution time, kernel matrix construction time, and classifier-head time. This path is useful because it separates circuit execution cost from trainable quantum-parameter optimization.

Expected bottleneck. The kernel path can become expensive because the kernel matrix grows with the number of samples. Therefore, QARCHGAUGE must report both sample scaling and feature-dimension scaling.

Kernel accounting rule. The kernel path is counted at the application level, not only at the circuit-kernel level. QARCHGAUGE records how many pairwise circuit evaluations are required, how many are actually executed, whether symmetry is used, and how the resulting matrix is consumed by the classifier. This matters because a circuit that is fast for one encoded sample can still be expensive after all sample pairs are evaluated. The design therefore exposes the sample-count multiplier instead of hiding it inside one aggregate runtime.

3) QNN/VQC Path: **Parameterized Circuit Execution.** The QNN/VQC path maps the same PCA features into a parameterized quantum circuit. It then updates trainable parameters through a classical optimizer. Each optimizer step can require many circuit evaluations.

Cost breakdown. QARCHGAUGE records ansatz type, depth, gate count, number of parameters, optimizer iterations, observable extraction time, and accuracy. This path is expected to be more expensive than the quantum kernel path because training repeatedly evaluates the circuit.

Why this matters. QNN/VQC models are often discussed as quantum machine-learning candidates. However, their systems

cost is easy to underestimate. QARCHGAUGE makes the repeated circuit evaluations visible.

Optimizer rule. For variational paths, QARCHGAUGE records the outer-loop optimizer separately from circuit execution. The record includes the number of parameter vectors, the number of observables, the number of shots per evaluation, and the measured time spent outside `cuQuantum`. This separation is needed because future quantum hardware can reduce circuit execution time without removing the classical optimizer loop. If the optimizer or data movement dominates, the projected hardware speedup has limited effect.

D. Measurement and Threshold Analysis

JSON record. Every run emits a JSON record. The record includes dataset metadata, model metadata, runtime breakdowns, accuracy, circuit qubits, circuit depth, gate counts, shots, GPU metadata, and Slurm metadata where available.

Summary path. QARCHGAUGE keeps raw case records separate from derived summaries. A sweep first writes one JSON object per case or per task shard. A summarizer then validates expected case counts, merges shards, computes medians and quantiles, and writes CSV/JSON summaries used by the figures. The paper audit reads these summaries and checks that the claims in the manuscript match the stored values. This design follows the same spirit as prior systems papers: the evaluation does not depend on hand-copied numbers.

Allocation accounting. QARCHGAUGE separates login-node smoke tests from Slurm performance runs. Login-node tests are only for correctness. Slurm runs must record elapsed time, queue time, requested walltime, allocated GPUs, and charged node-hours. This prevents short sanity checks from being mistaken for valid performance experiments.

1) Threshold Analysis: Execution model. Given circuit metadata, QARCHGAUGE estimates projected quantum execution time:

$$T_{\text{execute}} = \frac{N_s(D_1 t_1 + D_2 t_2 + D_m t_m)}{P_{\text{shots}}} + T_{\text{error}}, \qquad (5)$$

where N_s is the shot count, D_1 and D_2 are one- and two-qubit gate depths, D_m is measurement depth, t_1, t_2, and t_m are logical operation times, and P_{shots} is shot-level parallelism.

Projection scope. This model is a first-order lower bound, not a validated fault-tolerant schedule. The default sweep reports a dimensionless speedup so that the result is not tied to one vendor stack. The decomposed form above is the hardware hook: a fault-tolerant version can replace t_i, P_{shots}, and T_{error} with a surface-code stack using code distance, cycle time, logical-error budget, magic-state cost, decoder latency, and control bandwidth [8], [9]. The effective P_{shots} must be bounded by physical devices, decoders, control links, queue slots, and finite circuit evaluations; it is not an arbitrary multiplier. If a workload only wins after setting these terms unrealistically low, the conclusion is not advantage; it is a hardware requirement.

Worked hardware instantiation. To make the abstraction concrete, the projection can be instantiated as $T_{\text{error}} = T_{\text{compile}} + T_{\text{map}} + T_{\text{route}} + T_{\text{decode}} + T_{\text{ms}} + T_{\text{io}} + T_{\text{cq}}$ and $t_i = k_i d \tau_c$, where d is surface-code distance, τ_c is cycle time, and k_i is the number of code cycles for the logical operation. For example, an illustrative lower-bound stack with $d = 25$, $\tau_c = 1\,\mu$s, $k_1 = 1$, $k_2 = 4$, $k_m = 1$, $5\,\mu$s decoder latency per measured layer, a finite magic-state budget T_{ms}, and $P^{\text{eff}}_{\text{shots}} = 10^4$ parallel shots maps each measured case to a concrete logical-runtime budget. Changing any of these parameters moves the same frontier rather than changing the native baseline.

Encoding and hybrid-loop rule. Data loading and algorithmic recovery are not free vertical moves on the frontier. If a QRAM or load-store quantum architecture reduces T_{io}, the case moves left; if richer encodings, deeper ansatz layers, or more optimizer steps improve quality, they also increase D_1, D_2, D_m, N_{eval}, or T_{cq}. QARCHGAUGE therefore treats quality recovery as a requirement axis and reports the circuit metadata needed to price a co-designed algorithmic change.

Break-even search. QARCHGAUGE inverts the model. Instead of assuming a quantum speedup, it asks what logical gate time, shot throughput, and error overhead are needed for:

$$T_{\text{qhw}} < T_{\text{native}}. \qquad (6)$$

This gives a hardware requirement rather than a vague claim of advantage. The requirement can be read as a design target. If a workload enters the advantage region only when P_{shots} is very large, the bottleneck is device-level or system-level parallelism. If it enters only when T_{error} is small, the bottleneck is the fault-tolerance and decoder stack. If it never enters until the quality gap closes, the bottleneck is not gate speed; it is algorithmic quality, noise resilience, or encoding.

Advantage frontier. QARCHGAUGE does not reduce the result to one speedup number. It sweeps hardware assumptions and reports the feasible region where the projected quantum path beats the native path at the same quality. This region is the advantage frontier. If no reasonable region exists, the workload is classified by the limiting factor: quality, encoding, shots, error overhead, or native baseline strength.

Frontier classification. After the sweep, each case is assigned a primary limiting factor. A case is speed-limited when quality is close enough but the required runtime improvement is large. It is quality-limited when even aggressive speedup does not help unless the output gap closes. It is encoding- or shot-limited when non-kernel overhead dominates the projection. It is native-dominated when a strong native baseline makes the required improvement much larger than the quantum path can plausibly recover. This classification turns the frontier into a diagnostic result instead of only a plot.

E. Advantage Claim Checklist

A practical quantum-advantage claim is accepted only if the native and quantum paths solve the same input, use the same quality target, include end-to-end application cost, and compare against the strongest valid native baseline available in the experiment. QARCHGAUGE then asks whether the remaining requirement is speed, shots, error/control overhead,

TABLE II: Measured workload paths.

Family	Native path	Circuit path	Quality target
ML	six classifiers; AMP/XGBoost gate	QKernel, QNN/VQC	accuracy gap
Chemistry	dense exact; sparse Lanczos	VQE-style energy circuit	energy gap
Optimization	exact, greedy, local search, annealing	QAOA-style circuit	objective gap
Simulation	dense exact; sparse Krylov	Trotter/Hamiltonian simulation	observable gap

quality recovery, or algorithmic change. This is stricter than reporting a fast circuit kernel, but it prevents a different input, a weaker native path, or a lower-quality quantum output from looking like advantage.

F. Workload Suite

Table II shows the measured workload suite. The digits workload is the controlled calibration case. The practical suite covers the application families that motivate many real-world quantum-computing claims: AI classification, molecular energy estimation, combinatorial optimization, and Hamiltonian simulation. Each row keeps the same comparison discipline: native path, quantum-circuit path, and quality target are fixed before runtime is interpreted.

IV. Evaluation

This section evaluates QArchGauge in the same style as our previous systems papers: first the setup, then end-to-end behavior, then component analysis, scaling, and sensitivity. The result is not a claim that quantum wins today. The result is a measured architecture model that explains how fast, how parallel, and how accurate the future quantum path must be to beat native HPC paths, and which quantum-system resource should be prioritized for each workload family.

A. Evaluation Setup

Hardware Specification. Experiments run on a Top-20 TOP500 leadership-scale HPC system. Login-node smoke tests validate correctness, while performance experiments run through Slurm. GPU runs use four NVIDIA A100 GPUs per node and record JSON outputs, Slurm metadata, elapsed time, and allocation information.

Benchmark. The controlled benchmark is the expanded `sklearn digits` sweep. It varies class pair, sample count, PCA dimension, circuit depth, and seed. The practical benchmark suite then expands the same measurement discipline to ML, chemistry, optimization, and simulation.

Baselines. Native paths are selected from workload-specific candidates. ML selects among six classifiers, optimization selects among exact and heuristic solvers, chemistry evaluates dense exact and sparse Lanczos candidates, and simulation evaluates dense exact and sparse Krylov candidates.

Feasibility. Short login runs are only correctness gates. Performance evidence must come from Slurm jobs with accounting records, raw JSON counts, and completed exit status. The completed campaign below separates calibration, application coverage, baseline stress, scaling, and timing stability.

Campaign Summary. Figure 4 shows the completed evidence used by the paper. The campaign starts with a 160-case controlled ML calibration, expands to a 3,552-case practical suite, stresses chemistry/simulation coverage with 116-case and 104-case gates, scales fixed work from 16 to 256 GPUs, adds a 7,104-case larger-workload gate on 256 GPUs, and finishes with 12 warmup-separated repeat trials. This organization separates correctness gates from performance evidence and avoids treating a short smoke test as a performance claim.

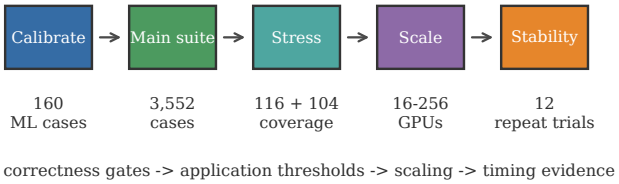
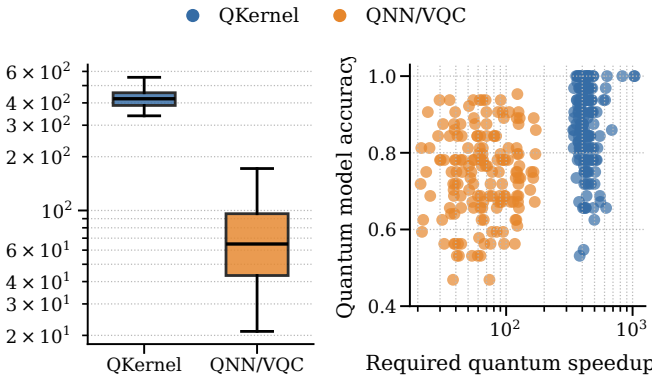


Fig. 4: Completed leadership-system evidence campaign. The figure separates correctness, application coverage, scaling, and timing evidence instead of compressing the evaluation plan into a table.



(a) Threshold distribution. (b) Quality vs. threshold.

Fig. 5: Expanded digits results over 160 cases. The quantum-kernel path often reaches useful accuracy but needs hundreds of times faster execution, while QNN/VQC has a lower threshold but usually lower quality.

Evaluation Questions. The evaluation asks six questions in order: how large the end-to-end threshold is, how quality changes the interpretation, whether the practical suite behaves like the controlled case, how much native-baseline strength moves the target, whether bundled multi-node runs scale, and which future-system resource each workload needs. The answers are carried mainly by figures and observation boxes; tables are kept only where exact numeric summaries are more readable than prose.

B. End-to-End Threshold Landscape

The expanded sweep contains 160 controlled digits cases over four class pairs, two sample counts, four PCA dimensions, multiple depths, and seeds. The digits circuits use one qubit per PCA feature dimension, so the controlled sweep uses 4, 8, 12, and 16 qubits. Figures 5a and 5b are generated from the measured case records and summarize threshold and quality versus the measured circuit configuration.

Data. Native ML remains strong across the workload, with native accuracy between 0.9219 and 1.0000. Quantum kernels sometimes reach high quality, with median accuracy 0.8750, but the measured circuit path still requires hundreds of times faster projected execution to match the native path. QNN/VQC has a smaller required speedup in many cases, but its median accuracy is 0.7500 and usually below the native baseline.

Interpretation. The figure shows why a runtime-only conclusion would be misleading. The QNN/VQC path is closer on time but worse on quality, while the quantum-kernel path is stronger on quality but farther on time.

Observation 1: The useful output is not "quantum is faster" or "quantum is slower." The useful output is the hardware requirement. On the controlled ML sweep, quantum kernels need a median 421.9× execution improvement to reach native time, while QNN/VQC needs only 64.9× but usually misses native quality. This separates a speed bottleneck from a quality bottleneck.

1) Quality Sensitivity: The threshold depends on the data pair. The easy pair, 0-vs-1, gives high median quantum-kernel accuracy at 0.9688, while harder pairs such as 3-vs-8 and 5-vs-8 reduce the median to 0.8125. QNN/VQC follows the same pattern but at lower quality: 0.8750 on 0-vs-1 and 0.6562 on 5-vs-8. Quality therefore changes the interpretation of speed. A lower runtime threshold does not imply advantage if the quantum model misses the native target quality.

2) Practical Application Landscape: The digits workload is useful because it is controlled, cheap, and easy to repeat. It is not enough for a practical quantum-advantage claim. Real users care about broader applications: AI pipelines, molecular energy estimation for drug discovery, optimization, and scientific simulation. These workloads have different native baselines and different quantum overheads.

QArchGauge also evaluates a practical suite beyond the controlled binary digits sweep. The suite covers ML classification, VQE-style chemistry, QAOA-style optimization, and Hamiltonian simulation. These workloads represent the application classes where users often expect quantum advantage, but the measured result shows that the threshold is application dependent and usually large. The main practical-suite result below uses the large 3,552-case profile on 128 GPUs. ML selects among six native classifiers, and optimization selects among exact, greedy, local-search, and annealing baselines. Chemistry and simulation use native numerical baselines on the same Hamiltonian instances and compare them with quantum-circuit executions simulated through `cuQuantum`.

Across 3,552 cases, native execution remains extremely fast: the median native path is 2.09 ms for ML, 0.170 ms for chemistry, 0.024 ms for optimization, and 2.08 ms for simulation. The simulated circuit path is roughly seven seconds per case. This creates median required speedups of 3,726.4× for ML, 42,491.4× for chemistry, 287,045.6× for optimization, and 3,071.0× for simulation. These numbers are not a final hardware claim; they are the measured input to the projection model.

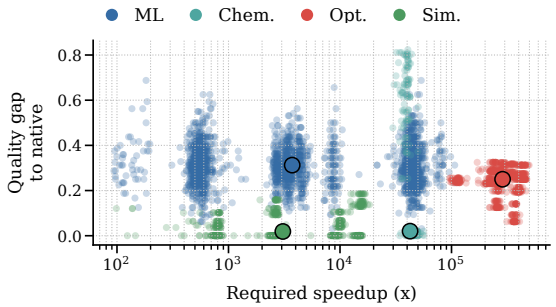
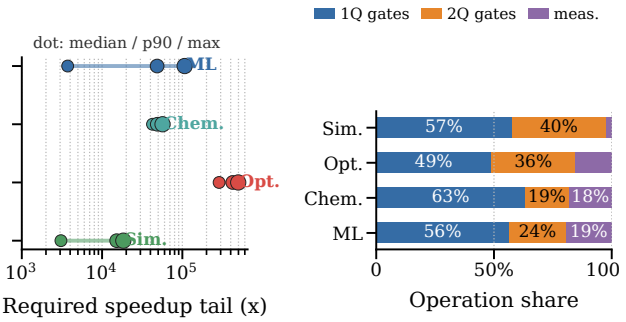


Fig. 6: Large practical-suite landscape over 3,552 cases. The plot keeps both axes that matter for advantage: required speedup and quality gap. A workload is not close to advantage unless it moves left in runtime and down in quality gap.



(a) Threshold tail. (b) Operation mix.

Fig. 7: Tail and circuit pressure by workload. The left plot shows median, 90th percentile, and maximum threshold pressure; the right plot separates one-qubit, two-qubit, and measurement pressure after the fixed simulation floor is removed.

Circuit Data. Median records are ML: 7.3K ops, 224 evals, 7.3s; chemistry: 0.7K ops, 41 evals, 7.1s; optimization: 3.4K ops, 101 evals, 7.0s; and simulation: 0.2K ops, one eval, 6.9s. The quantum-runtime IQRs are narrow relative to the family differences in gate counts: ML is 6.97–7.74s, chemistry is 6.87–7.35s, optimization is 6.77–7.20s, and simulation is 6.72–7.06s.

Tail Data. The long tail is workload-specific: ML moves from 3.7K× median to 48.5K× at p90, optimization from 287.0K× to 424.2K×, chemistry stays high but tight at 42.5K–56.6K×, and simulation reaches 18.4K× at max. The largest ratios are usually native-fast cases combined with the fixed small-circuit simulation floor, not a single unusually slow circuit kernel.

Interpretation. The similar seven-second path is not interpreted as a future hardware latency. It is the measured end-to-end application floor of this small-circuit simulation stack, including Python orchestration, simulator setup, repeated circuit invocation, state extraction, and host-side objective logic. The operation mix explains which architectural lever would matter after that fixed floor is removed: a workload dominated by repeated circuit evaluations needs shot-level and scheduling parallelism, while a workload with fewer but heavier state-evolution circuits is more sensitive to logical execution speed and simulator/hardware kernel efficiency.

Observation 2: The practical-suite gap is not an ML-only artifact. Native paths often finish in microseconds to milliseconds, while the simulated quantum-circuit path takes about seven seconds per case. However, the architecture bottleneck differs by family: simulation and chemistry mostly need faster logical execution and shot throughput after quality is controlled; ML and optimization first need better circuit output quality.

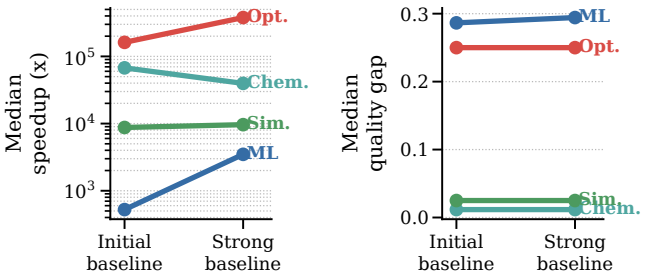
C. Component and Baseline Analysis

Observation 3: The native baseline is an architecture constraint, not bookkeeping. Strengthening the native side increases the ML threshold by 6.6× and the optimization threshold by 2.3×. A future quantum architecture therefore competes against the native software stack available at deployment time, not against a frozen toy baseline.

The strong-native run completed as a one-node, four-GPU bundled allocation on the leadership-scale system. The job finished 190 cases in 6 minutes and 59 seconds with exit code 0:0, 190 raw JSON outputs, and empty stderr logs. The selected ML baselines were RBF kernel ridge, nearest centroid, kNN, softmax regression, and linear ridge depending on the case; the selected optimization baselines were mostly greedy assignment, with exact enumeration selected when it was fastest at the best cut value.

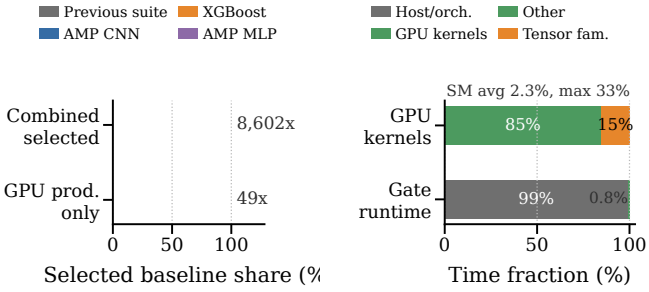
ML Production-native Gate. The main suite intentionally uses auditable small-task ML baselines because the quantum-circuit version is also constrained to small feature dimensions and 4–16 qubits. To test the reviewer concern directly, we add a 32-case production-style ML native gate over measured digits configurations with PyTorch AMP CNN/MLP candidates and XGBoost GPU-hist. The production candidates reach median quality 1.0000. Among production candidates, the AMP CNN is selected in 25 of 32 cases, XGBoost in five, and the AMP MLP in two. When these candidates are combined with the previous suite baselines, however, the selected native path remains the previous microsecond baseline in 24 cases and switches to the AMP CNN in eight. The median threshold changes from 8,876.8× to 8,601.6×; the production-only threshold is 49.3× because the GPU models are accurate but overhead-heavy for 8×8 digits. A larger ResNet-style ImageNet baseline would be the wrong task for this quantum-feasible input size; the measured gate answers the fairer question of whether a production GPU stack changes T_{native} on the same inputs.

Native Profiling Gate. We also profile the PyTorch AMP gate. Nsight Systems records 53 CUDA kernel rows, including 16 tensor-family kernels with `xmma`, `wmma`, `tensorop`, and `ampere_fp16` names. Tensor-family kernels account for 15.2% of GPU kernel time, but GPU kernels occupy 0.8% of the run, or 48.4 ms of a 6.07 s profiled gate. The remaining 99.2% is host orchestration, data movement, Python framework overhead, and process/file activity around very small training jobs. `nvidia-smi dmon` agrees with this interpretation: 30 one-second samples show 2.3% average SM utilization, a 33% peak, and no memory-bandwidth saturation signal. Nsight



(a) Runtime threshold shift. (b) Quality gap shift.

Fig. 8: Native baseline stress test. The slope view shows how each workload moves when the native path is strengthened, instead of hiding the shift inside grouped bars. Strengthening the native side makes the ML threshold 6.6× larger and the optimization threshold 2.3× larger than the initial native-baseline sweep.



(a) ML native gate. (b) Profiling gate.

Fig. 9: ML production-native and profiling gate. PyTorch AMP and XGBoost candidates reach native-level quality, but the small digits task remains dominated by previous microsecond baselines and by host orchestration in the profiled GPU gate.

Compute/Roofline counter collection was attempted, but the driver reported an unavailable profiling resource, likely due a site-level concurrent collector; the artifact records the error instead of fabricating Tensor-Core utilization. Therefore the paper claims tensor-family execution was observed, but it does not claim Roofline saturation.

Baseline Coverage Gate. We also run a separate accept-profile gate to strengthen the two application families that are easiest to criticize as toy baselines: chemistry and scientific simulation. The chemistry gate includes OpenFermion and PySCF-generated H_2, LiH, and H_2O active-space Hamiltonians, in addition to the built-in minimal H_2 and molecular-chain surrogate. The committed fixture set now includes LiH and H_2O active spaces at 4, 6, and 8 qubits; the 6–8 qubit fixtures pass a login-node smoke gate before larger allocation runs. The simulation gate compares dense exact evolution with sparse Krylov evolution on TFIM and Heisenberg Hamiltonians from 4 to 8 qubits.

Baseline Boundary. This stress test is not a claim that the native side is final domain SOTA or fully optimized for every accelerator feature. The added ML gate shows that PyTorch AMP and XGBoost candidates do not tighten this tiny digits threshold, but that result is specific to the same small inputs used by the quantum-circuit path. Tuned MILP and domain heuristics for optimization, tensor-network or projector-QMC

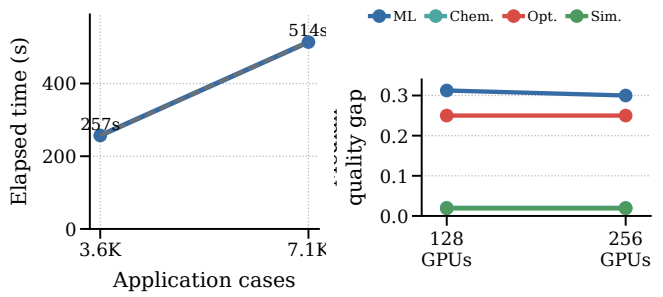
methods for simulation, larger ML datasets, and CCSD(T)-class chemistry references can still move T_{native} and quality targets. QArchGauge's rule is to report the strongest implemented, auditable native path, state its boundary, and expose how much the frontier moves when the native side is strengthened.

The strengthened chemistry and simulation baselines execute correctly in the same bundled one-node run. The job completed 116 cases in 6 minutes and 5 seconds with exit code 0:0; each of the four A100 tasks completed 29 cases, and stderr logs were empty. Chemistry reports a median 63,566.8× required speedup over 56 cases, and simulation reports 4,185.2× over 60 cases. For chemistry, both dense exact and sparse Lanczos/eigensolver candidates are evaluated for every case. The selected runtime baseline is dense exact for these small active-space Hamiltonians, but the run now uses OpenFermion and PySCF instances instead of only a hand-written surrogate. For simulation, sparse Krylov becomes the selected native method in 34 of 60 cases, especially at 6–8 qubits, showing that the native path is no longer a single dense solver.

Chemistry Active-space Coverage. We then run the expanded chemistry-only accept profile with the 4–8 qubit OpenFermion and PySCF fixture set. The molecular integrals and Jordan-Wigner Hamiltonians are generated once into JSON fixtures before the timed application comparison; each timed case starts from the same stored Pauli Hamiltonian for both native and quantum paths. Job 55515248 completed 104 chemistry cases on one GPU node in 5 minutes and 23 seconds with exit code 0:0, 104 raw JSON outputs, and empty stderr. The larger active-space subset includes LiH and H_2O at 6 and 8 qubits. LiH active-space cases require 21,847.9× at 6 qubits and 821.5× at 8 qubits; H_2O requires 21,613.0× at 6 qubits and 788.0× at 8 qubits. Dense exact remains the selected runtime baseline because these fixtures are still small, but sparse Lanczos is the best-quality native method in 62 of 104 cases. This result upgrades the chemistry evidence from a login smoke to a completed GPU coverage gate, while still reporting the limitation that these are active-space instances rather than production drug-discovery scale. The tolerance-scaled H_2O gaps exceed one, so those rows are useful failure evidence rather than near-quality advantage cases.

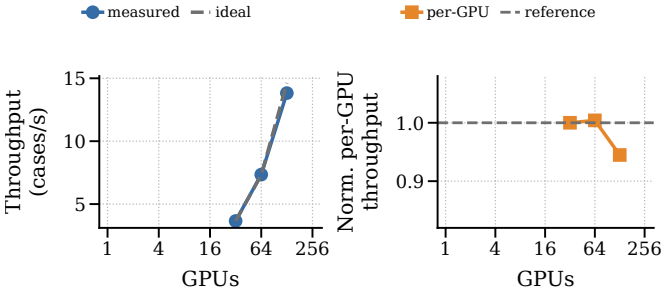
D. Leadership-scale Scaling Analysis

The large-profile suite contains 3,552 application cases. We evaluate two scaling modes and one larger-workload gate. In weak scaling, each GPU receives roughly 28 cases, so total work grows with the number of GPUs; this comparable-profile sweep covers 32–128 GPUs. In fixed-work scaling, every run executes the same 3,552 cases and the chunk slots are striped across the available GPUs from 16 to 256 GPUs. The separate 256-GPU larger-workload gate doubles the completed case count to 7,104 and checks whether time and quality remain interpretable at that scale. All modes use one Slurm task per A100 GPU and independent application cases; this is throughput scaling, not distributed single-circuit simulation.



(a) Work grows. (b) Quality remains stable.

Fig. 10: Larger-workload gate on 256 GPUs. The 256-GPU run completes 7,104 cases in 514 seconds with empty stderr logs. The larger run preserves the workload-level quality interpretation, so the extra cases increase evidence coverage rather than changing the advantage conclusion.

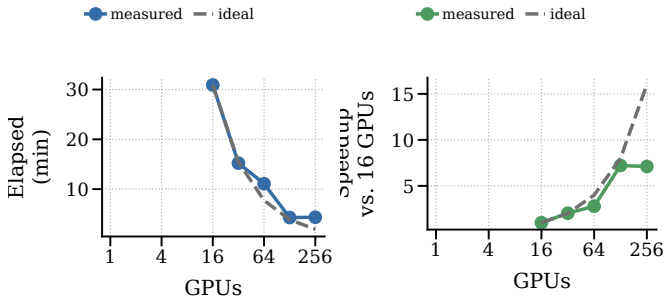


(a) Total throughput. (b) Per-GPU stability.

Fig. 11: Weak scaling of the comparable large-profile application suite. Total throughput grows nearly linearly from 32 to 128 GPUs while normalized per-GPU throughput remains close to the 32-GPU reference.

1) Weak Scaling: All comparable weak-scaling jobs completed with exit code 0:0. The generated raw JSON counts match the expected case counts: 896, 1,792, and 3,552. No failed case, traceback, or timeout was recorded in the task logs. The result is the desired throughput behavior for this workflow: total throughput grows with GPUs while the per-GPU rate stays close to stable. This matters because the advantage model is produced by many independent application cases, so throughput scaling determines how quickly the frontier can be mapped.

2) Strong Scaling: The fixed-work result measures time-to-solution for the same 3,552 cases. It reduces elapsed time from 30 minutes 55 seconds on 16 GPUs to 4 minutes 17 seconds on 128 GPUs, a 7.2× speedup over an 8× GPU increase. The 256-GPU run completes in 4 minutes 21 seconds, nearly identical to the 128-GPU result. This plateau is not a failure of `cuQuantum` and is not distributed single-circuit MPI synchronization: every Slurm task owns independent cases on one A100, and the workflow does not use inter-task circuit communication. At 128 GPUs the sweep still has about 28 cases per GPU; at 256 GPUs it has about 14 cases per GPU. With a seven-second per-case floor, stragglers, Python process orchestration, and task-local file output dominate before more GPUs can help. The ML profiling gate gives a representative



(a) Elapsed time. (b) Speedup.

Fig. 12: Strong scaling of the fixed 3,552-case application suite. The run reduces time-to-solution from 30 minutes 55 seconds on 16 GPUs to about 4 minutes 20 seconds on 128–256 GPUs, where task granularity and case imbalance become the limit.

small-case decomposition: GPU kernels occupy only 0.8% of that profiled run while host orchestration occupies 99.2%. We therefore diagnose the 128–256 GPU plateau as task granularity plus orchestration overhead, while still avoiding a stronger claim that we have a full 256-GPU Nsight Gantt trace.

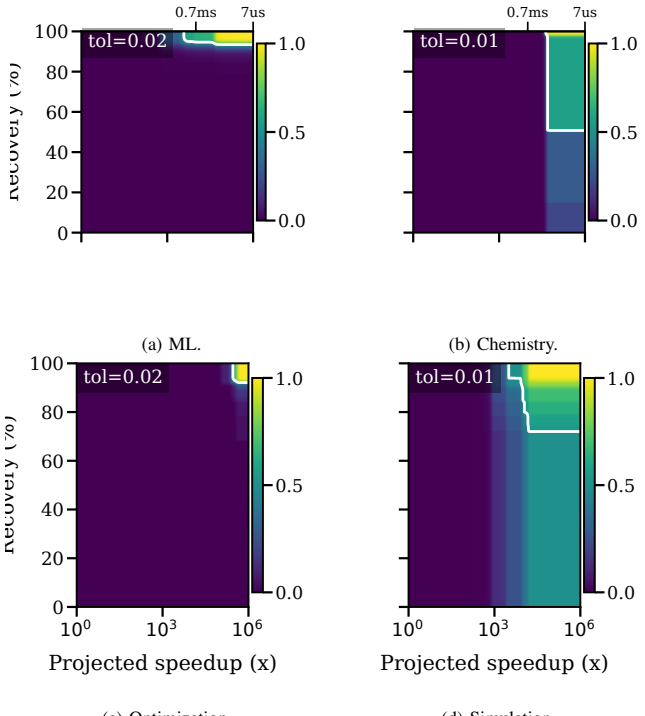
Observation 4: More GPUs help until the application sweep runs out of useful task granularity. Weak scaling remains healthy over the comparable 32–128 GPU profile, but fixed-work strong scaling saturates after 128 GPUs even though the 256-GPU larger-workload gate completes successfully. For this modeling workflow, architecture effort should improve per-case circuit execution and scheduling granularity, not only allocate more GPUs.

E. Advantage Frontier and Bottleneck Analysis

The practical-suite result gives one speed threshold per case, but a speed-only threshold is still incomplete. A practical advantage claim also needs comparable output quality. We therefore convert each measured case into an advantage frontier over two axes: projected quantum execution speedup and quality-gap recovery. A point is in the advantage region only if the projected quantum path is no slower than the selected native path and the remaining quality gap is within a small workload tolerance.

Observation 5: The frontier separates speed-limited and quality-limited workloads. Chemistry and simulation often have small quality gaps, so their frontier is mostly pushed right by runtime. ML and optimization require quality recovery before faster gates matter. Thus, the architecture target is a feasible region over speed and quality, not a single global speedup slogan.

Tolerance Sensitivity. The projection uses strict family tolerances: 0.02 for ML and optimization and 0.01 for chemistry and simulation. These values are model inputs, not universal domain standards. Figure 14 sweeps tolerance multipliers without rerunning GPU jobs. At 90% recovery, simulation at $10^4\times$ moves from 46.9% to 54.9% to 68.9% as the tolerance changes from $0.5\times$ to $1\times$ to $2\times$. ML at $10^5\times$ is much more tolerance-sensitive, moving from 0.8% to 9.8% to 83.8%. Chemistry at



(a) ML. (b) Chemistry. (c) Optimization. (d) Simulation.

Fig. 13: Advantage frontier over projected quantum speedup and quality-gap recovery. Color shows the fraction of measured cases in each family that would enter the advantage region under that hypothetical hardware and algorithmic improvement. The top-row secondary axis maps speedup to effective quantum execution time using the measured seven-second circuit path; the contour marks the 50% frontier when it appears.

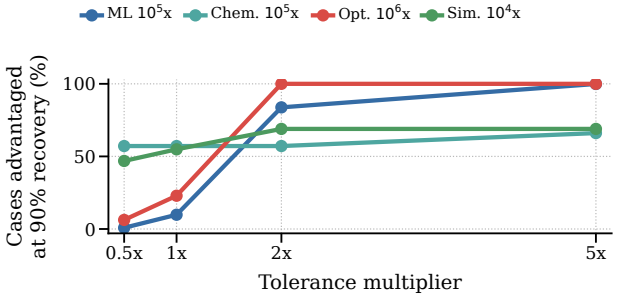


Fig. 14: Tolerance sensitivity of the advantage frontier. The same measured cases can be reinterpreted under stricter or looser quality budgets; this changes quality-limited ML and optimization much more than speed-bound chemistry.

$10^5\times$ is speed-bound under these tolerances, staying at 57.1% until the tolerance becomes very loose. Optimization remains hard unless both the speedup and quality budget are relaxed: at $10^6\times$, it moves from 6.2% at $0.5\times$ tolerance to 22.9% at $1\times$ and 100% at $2\times$. The raw projection artifact records the full grid so the analysis can be regenerated under other tolerances.

1) Hardware Projection Model: The projection model uses the measured native runtime, measured quantum-circuit simulation runtime, and measured tolerance-scaled quality gap for each case. For a projected quantum speedup S and quality-gap recovery R, a case enters the advantage region only if

S is at least the measured required speedup and the residual quality gap is below the workload tolerance. Formally, a case is counted as advantaged when

$$S \geq \frac{T_{qsim}}{T_{native}} \quad \text{and} \quad (1-R)\Delta Q \leq \epsilon_w,$$

where ΔQ is the measured quality gap in the workload's natural units and ϵ_w is the workload tolerance. The recovery parameter R is not a measured error-mitigation result; it is a requirement axis. Thus, a point on the frontier says how much algorithmic, mitigation, or fault-tolerance improvement would be needed before runtime speedup matters.

The frontier turns measured runs into concrete hardware targets. Simulation is the closest family: with 90% quality-gap recovery, a $10^4\times$ projected execution improvement covers 54.9% of measured simulation cases. Chemistry needs a larger runtime shift: at the same recovery level, $10^5\times$ covers 57.1% of chemistry cases. ML shows the opposite behavior. A $10^5\times$ speed improvement covers only 9.8% of ML cases at 90% recovery, but almost all cases if the quality gap is fully closed. Optimization is the hardest family in the current suite; even $10^6\times$ speedup covers only 22.9% of cases at 90% recovery and requires full quality recovery to cover all cases.

The speed axis is dimensionless in the figure, but it has a physical interpretation. Since the measured circuit path is about seven seconds per case, $10^4\times$, $10^5\times$, and $10^6\times$ correspond to roughly 0.7 ms, 70 μs, and 7 μs of effective end-to-end quantum execution per measured case before residual host and error terms. A fault-tolerant instantiation reaches that budget only if logical cycle time, code distance, decoder bandwidth, magic-state throughput, and effective shot parallelism jointly satisfy the decomposed model in Section III. Shot parallelism is a control and scheduling target: the useful rate is bounded by QPUs, decoders, controller fanout, queue policy, and available circuit evaluations. The frontier should therefore be read as a microarchitecture target, not a simulator speedup wish.

Observation 6: The frontier converts measurements into design guidance. Faster logical operations and shot-level parallelism are most valuable for simulation and part of chemistry. For ML and optimization, the priority shifts to quality-preserving encodings, error suppression, better ansatz structure, and reducing classical-quantum iteration overhead; faster gates alone do not create advantage.

2) Bottleneck Taxonomy: The main evaluation output is not that the simulator is slow. The simulator is the measurement tool. QArchGauge uses the measurements to explain what kind of quantum hardware and algorithmic behavior would be necessary for advantage. For each workload family, it reports whether the path is limited by speed, quality, encoding, shots, error overhead, or native-baseline strength.

Figure 15 shows this classification for the 3,552-case large run using the same family tolerances as the projection frontier. ML is quality-limited in all 2,048 cases because the quantum feature path does not match the selected native classifier accuracy. Optimization is also quality-limited in all 768 cases because the current QAOA grid has a large objective gap.

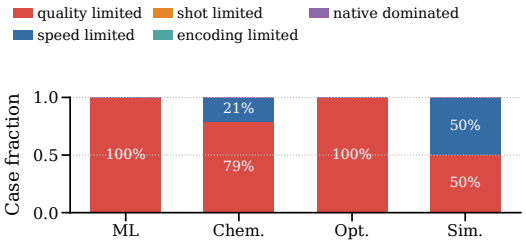


Fig. 15: Primary bottleneck taxonomy for the strong-native practical suite. ML and optimization are quality-limited in the current circuit configurations. Chemistry is mostly speed-limited, while simulation splits between speed and quality limits.

Chemistry has 48 speed-limited cases and 176 quality-limited cases. Simulation splits evenly, with 256 speed-limited cases and 256 quality-limited cases.

This classification is the practical insight. If a workload is speed-limited, faster logical gates or more shot parallelism may move it into the advantage region. If it is quality-limited, faster hardware alone is not enough; richer ansatzes, warm starts, parameter transfer, or noise-aware encodings must be priced as changes in depth, two-qubit pressure, optimizer evaluations, or classical-quantum feedback. If it is native-dominated, the native HPC path remains better unless the quantum algorithm moves along the frontier.

3) Sensitivity Scope: The expanded digits sweep covers class pair, sample count, PCA dimension, circuit depth, and seed; the practical suite adds workload family, native-baseline choice, chemistry grid size, circuit layer count, graph family, simulation model, and GPU count. These dimensions support the central claim that thresholds are workload-specific, while shot, ansatz, optimizer, mitigation, and repeated hardware trials remain explicit future sensitivity axes.

F. Operational Stability

The official practical sweep was first run as multiple shared-GPU jobs and then validated with one `salloc` allocation and one multi-task `srun`. The bundled pilot completed in 6 minutes and 54 seconds with 96 raw JSON outputs and empty stderr logs, and job 55516885 completed 12 measured warmup-separated trials in 58 seconds with exit code 0:0, empty stderr, and the maximum quantum-runtime CV is 0.0400. These gates support the timing evidence without replacing the large 3,552-case sweep.

V. Discussion and Threats to Validity

What the results prove. QArchGauge does not prove current quantum advantage. It proves a computer-architecture point: once task, quality target, and native baseline are fixed, advantage becomes a measured design-space boundary that says whether future effort should buy faster logical gates, more parallel shots, lower error/control overhead, stronger native co-design, or better algorithmic quality.

Baselines, workloads, and algorithms. The largest threat is an underpowered native baseline. We reduce it by selecting the fastest valid method among six ML classifiers, exact/heuristic

optimization, dense/sparse chemistry solvers, and dense/sparse simulation solvers, then measuring the strong-native shift. We also run a same-input ML production-native gate with PyTorch AMP CNN/MLP and XGBoost GPU-hist candidates; it reaches native-level quality but does not tighten the median threshold for the tiny digits cases because GPU framework overhead dominates. Stronger natives on larger tasks, including deeper CNN/ResNet families, boosted trees, tuned MILP/domain heuristics, tensor networks, projector QMC, and CCSD(T)-class chemistry references, can still move the frontier rightward. The suite is not exhaustive: chemistry/simulation use small active spaces, QAOA uses fixed grids, and 4–16 qubit measurements are not deployment-scale extrapolations; better ansatzes, encodings, and shots may improve recovery while increasing depth, evaluations, and host-QPU feedback.

Hardware projection and artifacts. `cuQuantum` is instrumentation, not the future hardware stack. Real hardware adds logical gate time, readout, finite shots, feedback, mapping, routing, data loading, queueing, and error correction; hardware-specific studies should replace T_{error} and P_{shots} with code distance, cycle time, logical-error budget, magic-state throughput, decoder latency, controller bandwidth, and device concurrency. The same record can carry energy: compare $E_{\text{native}} = N_{\text{GPU}} P_{\text{GPU}} T_{\text{native}}$ with a quantum budget that adds refrigerator, decoder FPGA/ASIC, control, and classical host power over T_{qhw}. We do not instantiate those watts without facility and decoder measurements, but the model makes a $10^5\times$ speed target testable against both time and energy. The scaling plateau is diagnosed from workflow structure, Slurm artifacts, and a representative Nsight/dmon gate showing 0.8% GPU-kernel time and 99.2% host-orchestration time on a small ML native gate. Nsight Compute counter collection failed because a driver profiling resource was unavailable; the artifact preserves that failure, so we do not claim Roofline saturation. Bundled Slurm runs, 12 warmup-separated trials, zero failures, maximum quantum-runtime CV is 0.0400, and artifact audits reduce timing and provenance risk.

VI. Related Work

Quantum Simulation and NISQ Systems. Computer architecture uses measurement and modeling to study immature systems [10]; QArchGauge applies that discipline to quantum systems. ScaleQsim, AURORA-Q, and `cuQuantum` improve state-vector simulation through distribution, GPUs, tiered memory, asynchronous movement, and optimized primitives [1], [2], [7]; NISQ studies show current-device limits [11]. SupermarQ, QASMBench, MQT Bench, QAOAKit, QED-C, and QCHALLenge provide circuits, metrics, parameters, and application-aware tests [3]–[6], [12], [13]. QArchGauge asks a different question: what hardware and quality threshold lets a quantum-circuit application beat a native HPC application?

Quantum Applications and Native Baselines. Quantum kernels, variational classifiers, VQE chemistry, QAOA optimization, and Hamiltonian simulation are common practical-advantage candidates [14]–[19], but each domain also has strong native baselines and different quality metrics [20]. Fault-tolerant and resource-estimation work shows that logical cost depends on code distance, cycle time, layout, magic-state factories, decoder latency, and physical assumptions [9], [21], [22]; hybrid runtimes add queue, fidelity, and classical-resource constraints [23], [24]. QArchGauge supplies the same-task baselines, quality targets, and threshold artifacts these studies need.

VII. Conclusion

In this paper, we present QArchGauge for same-input, same-quality quantum-advantage modeling. The controlled digits sweep requires $421.9\times$ for quantum kernels and $64.9\times$ for QNN/VQC, and the practical suite scales to 3,552 cases. With 90% quality-gap recovery, $10^4\times$ covers 54.9% of simulation cases and $10^5\times$ covers 57.1% of chemistry cases. The main lesson is to report an advantage frontier and architecture bottleneck taxonomy, not a yes/no slogan.

References

[1] C. Kim, E. Sohn, S. Kim, A. Sim, K. Wu, H. Tang, Y. Son, and S. Kim, "ScaleQsim: Highly Scalable Quantum Circuit Simulation Framework for Exascale HPC Systems," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3771577

[2] C. Kim, A. Sim, K. Wu, H. Tang, Y. Son, J. Park, and S. Kim, "AURORA-Q: Asynchronous Unified Resource Optimizer for Quantum Simulation on HPC System," Accepted to the IEEE International Conference on Distributed Computing Systems, 2026, accepted for publication.

[3] T. Tomesh, P. Gokhale, V. Omole, G. S. Ravi, K. N. Smith, J. Viszlai, X.-C. Wu, N. Hardavellas, M. R. Martonosi, and F. T. Chong, "SupermarQ: A scalable quantum benchmark suite," in *2022 IEEE International Symposium on High-Performance Computer Architecture*, 2022, pp. 587–603.

[4] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.

[5] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[6] R. Shaydulin, K. Marwaha, J. Wurtz, and P. C. Lotshaw, "QAOAKit: A toolkit for reproducible study, application, and verification of the QAOA," in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software*, 2021, pp. 64–71.

[7] NVIDIA, "NVIDIA cuQuantum SDK Documentation," https://docs.nvidia.com/cuda/cuquantum/, 2026, accessed July 3, 2026.

[8] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, no. 3, p. 032324, 2012.

[9] D. Litinski, "A game of surface codes: Large-scale quantum computing with lattice surgery," *Quantum*, vol. 3, p. 128, 2019.

[10] K. Skadron, M. Martonosi, D. I. August, M. D. Hill, D. J. Lilja, and V. S. Pai, "Challenges in computer architecture evaluation," *Computer*, vol. 36, no. 8, pp. 30–36, 2003.

[11] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[12] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, "Application-oriented performance benchmarks for quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 4, pp. 1–32, 2023.

[13] M. Erdmann, L. Karch, A. Awasthi, C. I. Jones, P. Bhardwaj, F. Krellner, J. Stein, C. Linnhoff-Popien, N. Kraus, J. P. Eder, S. Braun, and T. Liu, "Quantum computing—strategic recommendations for the industry," 2026.

[14] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[15] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles, "Challenges and opportunities in quantum machine learning," *Nature Computational Science*, vol. 2, no. 9, pp. 567–576, 2022.
[16] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.
[17] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014.
[18] I. M. Georgescu, S. Ashhab, and F. Nori, "Quantum simulation," *Reviews of Modern Physics*, vol. 86, no. 1, pp. 153–185, 2014.
[19] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer, and P. Zoller, "Practical quantum advantage in quantum simulation," *Nature*, vol. 607, no. 7920, pp. 667–676, 2022.
[20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[21] W. van Dam, M. Mykhailova, and M. Soeken, "Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation," 2023.
[22] M. Webber, V. Elfving, S. Weidt, and W. K. Hensinger, "The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime," *AVS Quantum Science*, vol. 4, no. 1, p. 013801, 2022.
[23] S. Pehlivanoglu, U. de Muelenaere, P. Kogge, and A. Sabry, "Qurator: Scheduling hybrid quantum-classical workflows across heterogeneous cloud providers," 2026.
[24] M. Tejedor, M. Grossi, C. Tüysüz, R. Rocha, and S. Vallecorsa, "Kubernetes-orchestrated hybrid quantum-classical workflows," 2026.

AI USE

OpenAI Codex assisted with drafting, editing, code generation, artifact consistency checks, and LaTeX build debugging. The authors are responsible for the final technical content, experiments, figures, citations, and claims.